

CSS Box Model

Master Spacing, Sizing, and Layout

■ BOX MODEL

■ LEARNING OUTCOME

By the end of this module you will fully understand the CSS box model, use border-box sizing correctly, control margin, padding, and border, understand display properties, and centre elements confidently.

■ Skills You Will Gain

- Explain the four box model layers: content, padding, border, margin
- Apply box-sizing: border-box globally
- Use shorthand notation for margin and padding
- Understand display: block, inline, inline-block, none, flex, grid
- Centre elements horizontally and vertically with multiple methods
- Use overflow, visibility, and opacity correctly

■ Every layout bug you will ever face is a box model bug. Understanding how browsers calculate size and spacing is the single most important CSS concept to master.

SECTION 1

The Four Layers of the Box Model

Every HTML element is a box with 4 layers

1. CONTENT: The actual content — text, image, etc. Width and height apply here by default. 2. PADDING: Space INSIDE the element, between content and border. Takes the element's background colour. 3. BORDER: A line around the padding. Has width, style, and colour. 4. MARGIN: Space OUTSIDE the element. Always transparent — cannot be coloured. Default (content-box): width/height = content only. Total = content + padding + border + margin Better (border-box): width/height = content + padding + border. Total = width + margin

```
/* ===== GLOBAL RESET — Apply border-box to everything ===== */ *, *::before, *::after {
box-sizing: border-box; /* Include padding + border in width/height */ } /* Now width: 300px MEANS
total width is 300px */ .card { width: 300px; /* Total width = 300px (border-box mode) */ padding:
24px; /* Inside space — takes background colour */ border: 2px solid #E44D26; margin: 16px; /*
Outside space — transparent */ background: white; border-radius: 12px; } /* Margin shorthand:
margin: all; (4 equal sides) margin: top-bottom left-right; (2 values) margin: top left-right
bottom; (3 values) margin: top right bottom left; (4 values — clockwise) */ .example { margin: 0
auto; /* top/bottom: 0, left/right: auto — centres block element */ padding: 16px 24px; /*
top/bottom: 16px, left/right: 24px */ } /* Individual sides */ .card-content { padding-top: 16px;
padding-right: 24px; padding-bottom: 16px; padding-left: 24px; /* Same as: padding: 16px 24px; */ }
```

SECTION 2

Borders and Outlines

```
/* Border shorthand: width style colour */ .box { border: 2px solid #333; /* all 4 sides */
border-top: 3px dashed red; /* individual side */ border-bottom: none; /* remove one side */
border-radius: 8px; /* rounded corners */ border-radius: 50%; /* perfect circle (if equal width &
height) */ border-radius: 16px 4px; /* top-left/bottom-right top-right/bottom-left */ } /* Border
styles */ border-style: solid | dashed | dotted | double | groove | ridge | none | hidden; /*
OUTLINE — different from border: */ /* - Does NOT take up space (doesn't affect layout) */ /* -
Drawn outside the border */ /* - Cannot have individual sides */ .focusable:focus { outline: 2px
solid #E44D26; /* never remove outline — it's accessibility! */ outline-offset: 2px; /* space
between element and outline */ } .no-outline-visual:focus { outline: none; /* ONLY remove if you
provide an alternative focus style */ box-shadow: 0 0 0 3px rgba(228,77,38,.4); /* custom focus
ring */ }
```

SECTION 3

Display Property

Display Value	Behaviour
block	Takes full available width. Forces new line before and after. Respects width, height, margin, padding on all sides.
inline	Only as wide as content. Flows with text. Ignores width, height, vertical margin. Horizontal padding/margin work.
inline-block	Like inline (flows with text) but respects width, height, and all margin/padding. Best for buttons and nav items.

none	Element is completely removed from layout AND from accessibility tree. Use hidden attribute for semantic hiding.
flex	Block-level flex container. Enables Flexbox layout for direct children.
inline-flex	Inline flex container — behaves like inline-block but children use flexbox.
grid	Block-level grid container. Enables CSS Grid layout for direct children.
contents	Element itself not rendered, but children are. Useful for semantic wrappers.

SECTION 4

Overflow, Visibility, and Opacity

```
/* overflow: what to do when content is larger than the box */ .box { overflow: visible; /*
DEFAULT: content flows outside box */ overflow: hidden; /* clip content at box boundary */
overflow: scroll; /* always show scrollbars */ overflow: auto; /* scrollbars only when needed */ /*
Different for each axis */ overflow-x: hidden; overflow-y: auto; /* Text truncation with ellipsis
*/ white-space: nowrap; overflow: hidden; text-overflow: ellipsis; /* shows "..." when text is
clipped */ } /* visibility vs display: none */ .hidden-display { display: none; } /* removed from
layout, accessibility, and flow */ .hidden-visibility { visibility: hidden; } /* INVISIBLE but
still takes up SPACE */ /* opacity — affects element AND all its children */ .faded { opacity: 0.5;
} /* 50% transparent — children too */ .invisible { opacity: 0; } /* invisible but still takes space
and is interactive */ /* For just the background, use rgba instead */ .semi-bg { background:
rgba(228, 77, 38, 0.5); } /* only bg is transparent, text stays opaque */
```

SECTION 5

Centering Techniques

Method	Code	Use Case
Margin auto (horizontal)	width:300px; margin:0 auto;	Centre block element horizontally
Flexbox centre	display:flex; justify-content:center; align-items:center;	Centre in container — most versatile
Grid place-items	display:grid; place-items:center;	Centre single item — cleaner than flex
Absolute + transform	position:absolute; top:50%; left:50%; transform:translate(-50%,-50%);	Centre in positioned parent
Inline/text-align	text-align:center;	Centre inline content and text
Margin auto (both)	margin:auto; (inside flexbox/grid)	Centre flex/grid item in both axes

■ PRACTICE Q & A

Q1. What is the difference between box-sizing: content-box and border-box?

A: *content-box (default): width/height = content only. Total width = content + padding + border. border-box: width/height = content + padding + border. What you set is what you get. Always use border-box — it's far more predictable.*

Q2. What is the difference between margin and padding?

A: *Margin: space OUTSIDE the element, always transparent, collapses between adjacent block elements. Padding: space INSIDE the element, takes the element's background colour, never collapses.*

Q3. What is margin collapse and when does it happen?

A: When two vertical margins touch (adjacent siblings, parent-child without padding/border), the larger margin wins instead of adding. Only affects vertical (top/bottom) margins. Doesn't happen with flexbox or grid children.

Q4. What is the difference between `display:none` and `visibility:hidden`?

A: `display:none` removes the element from layout — no space reserved, not in accessibility tree. `visibility:hidden` hides the element but PRESERVES the space it occupied. The element is still in the accessibility tree.

Q5. Why should you never remove outline for focused elements?

A: Outline is the focus indicator — essential for keyboard navigation accessibility. Users who navigate with keyboard (including people with motor disabilities) cannot see where they are on the page without focus styles. Provide an alternative style if removing the default.

■ Box Model Cheat Sheet	
<code>*, *::before, *::after { box-sizing: border-box; }</code>	Global border-box reset
<code>padding: T R B L</code>	Inside space – clockwise shorthand
<code>margin: 0 auto</code>	Centre block element horizontally
<code>margin: T R B L</code>	Outside space – clockwise shorthand
<code>border: 2px solid #333</code>	Border shorthand: size style colour
<code>border-radius: 50%</code>	Circle when width = height
<code>display: block</code>	Full width, new line
<code>display: inline-block</code>	Flows inline, respects size
<code>display: none</code>	Remove from layout completely
<code>visibility: hidden</code>	Invisible but keeps space
<code>overflow: hidden</code>	Clip overflowing content
<code>text-overflow: ellipsis</code>	Show ... for clipped text